Homework Assignment #1
Database Systems – Semester A 2026/2027

Due 12/10/2026

This assignment consists of two parts:

- **Part A** – This part is technical, and you need to follow the instructions in order to set up the Sakila database on which you will run your queries.
- **Part B** – Contains ten questions that you are required to answer and submit on Lemida.

Part A – Installing the Sakila Database

Before you start

- Make sure you followed Exercise 0 before this exercise.
- Make sure your MySQL server (the Docker container from class) is running, and that you can connect to it from MySQL Workbench.

About the database

Sakila is the official MySQL sample database. It models a DVD rental business: films, actors, categories, stores, an inventory of copies, customers, rentals and payments.

Step 1 – Download

1. Download the Sakila archive from the official MySQL site:
   `https://downloads.mysql.com/docs/sakila-db.zip`
   (it is also listed under *Example Databases* at `https://dev.mysql.com/doc/index-other.html`).
2. Extract the archive. You will get a folder named `sakila-db` with three files:
   - `sakila-schema.sql` – creates the schema, tables, views, procedures and triggers.
   - `sakila-data.sql` – inserts the data.
   - `sakila.mwb` – a Workbench model (ER diagram). You don't need it for the import, but you can open it to look at the schema.

Step 2 – Import in MySQL Workbench

The order matters: **first the schema, then the data.**

1. In Workbench, open your connection to the MySQL server.
2. Click **File** → **Open SQL Script...** and select `sakila-schema.sql`. The script opens in a new query tab.
3. Run the whole script.
4. Repeat with `sakila-data.sql`.
5. Refresh the schemas in MySQL workbench to see the Sakila schema.

Step 3 – Verify the installation

Run the following in a new query tab:

```
USE sakila;
SHOW TABLES;
```

`SHOW TABLES` should return 23 rows.

Tables you will need in this exercise

- `film`
- `actor`
- `film_actor`
- `category`
- `film_category`
- `inventory`
- `rental`
- `customer`
- `payment`

Part B – SQL Queries

- **If the zip file is named incorrectly, or the `details.txt` ID file is missing, 5 points will be deducted from the grade for each.**
- Your final submission should contain a zip file containing all submission requirements and a `details.txt` file that should contain the ID and name of each submitting student, such as:

  ```
  first_name last_name 123456789
  first_name last_name 987654321
  ```

- The zip file should be named with your ID, such as `"123456789.zip"`. If multiple students are submitting, then separate the ID numbers with underscore, such as `"123456789_987654321.zip"`.
- In this assignment, your zip file needs to contain **10 .py files**, and the `details.txt` file. Make sure you don't add any other unnecessary files to the zip.
- Submission is through the course website and can be either individually or in pairs – if you submit in pairs only one of the two should submit and that student will also be the one to receive feedback. A submission for three people is not permitted, if you are unable to find a pair then you should submit by yourself (it is not necessary to ask for permissions to submit by yourself).

Data

All the questions in this part will assume the Sakila database from Part A.

Requirements

For each question you are required to submit a single file called `q{n}.py` (for example `q1.py`, `q2.py` etc.) that will include the following:

- The SQL query you used.
- Documentation – **mandatory**. Use `#` to add comments.
- Each of the python files need to be in this format:

```
import mysql.connector

if __name__ == '__main__':
    mydb = mysql.connector.connect(
        host="localhost",
        user="root",
        password="root",
        database="sakila",
        port="3307",
    )

    cursor = mydb.cursor()
    cursor.execute("""
        PUT_YOUR_QUERY
    """)
    print(', '.join(str(row) for row in cursor.fetchall()))
```

```
cursor.close()
mydb.close()
```

Important Notes

- The format is important because the testing is automatic. If the automatic script fails because you changed the format, you will not get any points for the question.
- Return **exactly** the columns requested, in the requested order and with the requested labels (use `AS`), and sort exactly as requested.
- Do not use commands that have not yet been learned, or will not be learned at all - a query that uses such commands, **all points will be deducted**.
- You should assume that the data we are using is different from yours, do not make any assumptions on the data.
  - For example, if we ask for the customers who paid the most, do not write a query that contains their names or IDs because you already know who they are. We might use an altered version of the data, where the answer is different.

Questions

1. List all films rated `'PG-13'` whose length is **between 100 and 120 minutes (both included)** and whose rental rate is **strictly below 3.00**. Sort the result by **length in descending order**; films of the same length should be sorted by **title alphabetically**.
   This query should return four columns: `title`, `rating`, `length`, `replacement_cost`.

2. List, **without duplicates**, the first names of all **active** customers (`active = 1`) whose first name consists of **exactly four characters** and whose **second character is 'A'**. Sort the names alphabetically.
   This query should return a single column: `first_name`.

3. Get all films whose **description contains the word 'Documentary'** and whose **special features include 'Commentaries'**. For these films, calculate:

   a. The number of films.
   b. The average length.
   c. The minimum replacement cost.
   d. The maximum replacement cost.

   This query should return a single row and four columns: `num_films`, `avg_length`, `min_cost`, `max_cost`.

4. For each film, compute the number of films in the category and their average length. Return only categories that contain **at least 60 films** and whose average film length is **strictly greater than 110 minutes**. Sort the result by the **average length in descending order**.
   This query should return three columns: `category`, `num_films`, `avg_length`.

5. Find the customers who made **at least 35 payments** and whose **total amount paid is at least 175**. For each such customer return the customer ID, first name, last name, number of payments and total amount paid. Sort the result by the **number of payments in descending order**; customers with the same total should be sorted by **customer ID in ascending order**.
   This query should return five columns: `customer_id`, `first_name`, `last_name`, `num_payments`, `total_paid`.

6. Two different films are called *twins* if they have the **same length**, the **same rental rate**, the **same replacement cost** and the **same rating**. Retrieve all pairs of twin films. Each pair should be returned **only once**, with `title1` appearing **before** `title2` alphabetically (for example, ⟨ACADEMY DINOSAUR, ZORRO ARK⟩ and not ⟨ZORRO ARK, ACADEMY DINOSAUR⟩). Sort the result by `title1`.
   This query should return six columns: `title1`, `title2`, `length`, `rental_rate`, `replacement_cost`, `rating`.

7. For each pairs of actors who appeared **together in at least 5 films** return each actor's full name (first name, a space, last name) and the number of films they share. Each pair should be returned **only once**, where `actor1` is the actor with the **smaller** `actor_id`. Sort the result by the **number of shared films in descending order**, then by `actor1` and then by `actor2` alphabetically.
   This query should return three columns: `actor1`, `actor2`, `shared_films`.

8. The marketing team wants a single list of all the people in the database, **actors and customers**, whose **last name starts with 'WIL'**. A person that appears in both tables with the same first and last name should appear **only once**. Sort the result by `last_name` and then by `first_name`.
   This query should return two columns: `first_name`, `last_name`.

9. Find the customers who rented **at least one 'Horror' film rated** `'NC-17'` and **also at least one 'Music' film rated** `'G'`. Sort the result by `customer_id`.
   This query should return three columns: `customer_id`, `first_name`, `last_name`.
   Ignore the MySQL workbench warning if exists (if it runs it's ok)

10. Find the actors who appeared in **at least one 'Action' film** but **never appeared in a 'Drama' film**. Sort the result by `last_name`, then by `first_name`, and then by `actor_id`.
    This query should return three columns: `actor_id`, `first_name`, `last_name`.
    Ignore the MySQL workbench warning if exists (if it runs it's ok)